

Debugging

François Trahay



Debugging

But: comprendre l'exécution d'un programme

- Pourquoi le programme *crash* ?
- Pourquoi le résultat est 12 ? Ca devrait être 17, non ?
- Pourquoi le programme est bloqué ?

Debugging “manuel”

- On part d'un état (supposé) correct du programme
- Ajout de `printf("entrée dans foo(n=%d)\n", n);` dans le code source
- But: suivre le flot d'exécution et l'évolution des variables
- Avantages:
 - Facile à mettre en oeuvre
- Inconvénients
 - Beaucoup de code à modifier
 - Besoin de recompiler
 - Besoin de nettoyer le code après le debugging

Utilisation d'un Debugger

- Exécution du programme contrôlée par un debugger
- Permet:
 - d'examiner la valeur d'une variable
 - d'exécuter le programme en mode pas à pas
 - de mettre en pause le programme
 - d'insérer des *points d'arrêt*
- Exemple: GDB - The GNU Project Debugger

Exemple d'utilisation de GDB

```
$ ./sigsegu
Debut du programme
Erreur de segmentation

$ gdb ./sigsegu
[...]
(gdb) run
Starting program: ./sigsegu
Debut du programme

Program received signal SIGSEGV, Segmentation fault.
0x000000000040050b in main (argc=1, argv=0x7fffffffdd68) at sigsegu.c:7
7      *ptr=5;
(gdb) print ptr
$1 = (int *) 0x0
```

Utiliser GDB

- Pré-requis: compiler le programme avec l'option `-g`
 - Inclue les noms de fonctions/variables dans le binaire pour faciliter le débbugging
- Charger le programme dans gdb: `gdb ./programme`
- Lancer le programme: `r` (ou `run`)
- Arrêter le programme: `Ctrl+C`
- Quitter gdb: `q` (ou `quit`)

Examiner l'état du programme

La commande `bt` (ou `backtrace`)

- affiche la pile d'appel
- pour chaque niveau
 - *callsite*: emplacement de l'appel de fonction
 - possibilité d'inspecter les variables locales
- sélection de la *stack frame* `#x` avec la commande `frame [x]`.

```
(gdb) bt
#0  baz (a=2) at backtrace.c:7
#1  0x0000000000400581 in bar (n=5, m=3) at backtrace.c:15
#2  0x00000000004005ae in foo (n=4) at backtrace.c:21
#3  0x0000000000400559 in baz (a=5) at backtrace.c:9
[...]
```

Etat des variables d'un processus

- commande `p [var]` (ou `print [var]`)
 - affiche la valeur de `[var]`
- commande `display [var]`
 - affiche la valeur de `[var]` à chaque arrêt du programme
- `[var]` peut être une variable (eg. `p.n`)
- `[var]` peut être une expression (eg. `p.ptr->next->data.n`)

Exécution pas à pas

Une fois le programme lancé, possibilité d'exécuter les instructions une par une:

- **n** (ou **next**) : exécute la prochaine instruction, puis arrête le programme.
- **s** (ou **step**) : idem. Si l'instruction est un appel de fonction, ne descend pas dans la fonction.
- **c** (ou **continue**) : continue l'exécution du programme (arrête le mode pas à pas)

Points d'arrêt

- *“Arrête le programme dès qu’il atteint cette ligne de code”*
 - `b fichier.c:123`
- Permet d’examiner l’état du programme à certains points (exemple: entrée de la fonction buggée)

Surveiller une variable

- “Arrête toi dès qu’on modifie la variable $[x]$ ”
 - `watch x`
- Permet de trouver les endroits où une variable est modifiée
 - “Mais `ptr` ne devrait pas être **NULL** ! Qui a fait ça ?”

Valgrind

- Outils de débugging et de profilage
 - Détection de fuites mémoire
 - Utilisation de variables non initialisées
- `valgrind [programme]`

Sanitizers

Outre valgrind, vous pouvez utiliser des *sanitizers* pour repérer des soucis dans votre code:

- **AddressSanitizer**: détecte les accès hors-tableau, et l'utilisation des pointeurs après la libération
 - Compilez et linkez votre code avec `-fsanitize=address`
- **Undefined**: détecte divers comportements non-définis, notamment avec les pointeurs
 - Compilez et linkez votre code avec `-fsanitize=undefined`
- **Memory**: détecte des accès à de la mémoire non initialisée
 - Compilez et linkez votre code avec `-fsanitize=memory`

Pointeurs de fonction

- Toute fonction a une adresse
 - `foo` désigne l'adresse de la fonction `int foo(int a, char b);`
 - l'adresse correspond à l'endroit où est situé le code dans la mémoire
- Un pointeur de fonction¹ désigne l'adresse d'une fonction
- Exemple de déclaration:
 - `int (*function_ptr)(int a, char b) = foo;`
- Utilisation:
 - `function_ptr(12, 'f');` // appelle la fonction `foo`

¹ Oui, c'est sans rapport avec le debugging, mais pour équilibrer les séances, nous avons préféré ne pas aborder cette notion lors du cours sur les pointeurs :-)